

Examen de práctica: Git, bash y Docker

CU

Nombre

Dura **45 minutos**. Sin apuntes, formularios ni dispositivos electrónicos: sólo estas hojas, pluma o lápiz y goma. Contesta en el espacio debajo de cada pregunta.

En los parciales cada herramienta se preguntó sola. Aquí van juntas: el script vive en Git, el Dockerfile lo copia, y lo que imprime el contenedor depende de qué ramas mezclaste y de qué había en tu disco al construir. Todo se comprobó corriendo el escenario completo con Git 2.34, bash 5.2 y Docker 29.6.

La idea que atraviesa el ejercicio: **Git, bash y Docker no se avisan entre sí**. Git guarda commits, docker build copia lo que hay en tu disco, y bash corre lo que le den.

El repo

reporte/ cuenta ventas. En main hay un solo commit, **B**, con esto:

```
reporte/      ← estás aquí
├─ Dockerfile
├─ contar.sh
├─ datos/
│  └─ ventas.csv
```

contar.sh, con sus números de línea:

```
1  archivo="$1"
2  if [[ ! -f "$archivo" ]]; then
3      printf 'no existe: %s\n' "$archivo" >&2
4      exit 1
5  fi
6  printf 'filas: %s\n' "$((wc -l < "$archivo"))"
```

Dockerfile (la imagen bash:5.2 es Alpine con bash instalado):

```
FROM bash:5.2
WORKDIR /r
COPY contar.sh .
CMD ["bash", "contar.sh", "datos/ventas.csv"]
```

datos/ventas.csv, cuatro renglones:

```
MX,10
US,5
MX,7
CA,3
```

Tres personas crearon una rama cada una, todas desde B, con un commit cada una:

```
+---F      filtro
|
+---T      titulo
|
+---I      imagen
|
B          main
```

Rama	Commit	Qué cambia
filtro	F	contar.sh, línea 6: <code>wc -l < "\$archivo"</code> pasa a <code>grep -c MX "\$archivo"</code>
titulo	T	contar.sh, línea 6: <code>'filas: %s\n'</code> pasa a <code>'total: %s\n'</code>
imagen	I	Dockerfile: agrega <code>COPY datos/</code> después de <code>COPY contar.sh .</code>

`grep -c MX` cuenta los renglones que contienen MX; `wc -l` cuenta todos.

Parte 1 · Antes de mezclar nada

Estás en main, con B tal cual.

a · El script en tu máquina

¿Qué imprime `bash contar.sh datos/ventas.csv`?

b · Un archivo que no existe

¿Qué imprime `bash contar.sh datos/mayo.csv`, y qué imprime justo después `echo $??`? ¿Por qué ese número?

c · La imagen sin datos

Corres `docker build -t rep:0 .` y luego `docker run --rm rep:0`. ¿Qué imprime, y por qué, si `datos/ventas.csv` sí existe en tu carpeta?

d · Hazlo funcionar sin reconstruir

Escribe un `docker run` que haga funcionar `rep:0` sin reconstruir la imagen. ¿Qué imprime?

Parte 2 · La secuencia

Los comandos se corren **en orden**, todos desde reporte/ y empezando en main. Cada fila parte de lo que dejó la anterior.

Fila 1 · ¿Fast-forward o commit de merge?

```
git merge filtro
```

Fila 2 · ¿Y ahora? ¿Por qué es distinto?

```
git merge imagen
```

Fila 3 · ¿Qué imprime el run?

```
docker build -t rep:1 .  
docker run --rm rep:1
```

Fila 4 · ¿Qué imprime cada run? (dos casillas)

```
echo "MX,1" >> datos/ventas.csv  
docker run --rm rep:1  
docker run --rm -v "$(pwd)/datos":/r/datos rep:1
```

Fila 5 · ¿Qué responde Git?

```
git merge titulo
```

¿Te deja intentarlo aunque datos/ventas.csv tiene un cambio sin commit?

Fila 6 · ¿Cómo se ve el archivo? ¿Cuál lado es HEAD?

```
cat contar.sh
```

Escribe cómo se ven ahora las últimas líneas.

Fila 7 · ¿El build? ¿El run? ¿El código?

```
docker build -t rep:2 .
docker run --rm -v "$(pwd)/datos":/r/datos rep:2
echo $?
```

Fila 8 · Resuelve quedándote con los dos cambios

```
# editas contar.sh y resuelves el conflicto
git add contar.sh
git commit
```

Escribe la línea 6 resuelta. ¿Qué muestra después `git status --short`?

Fila 9 · ¿Qué sale de la caché?

```
docker build -t rep:3 .
```

Fila 10 · ¿total: 2 o total: 3?

```
docker run --rm rep:3
```

¿Por qué, si el cambio de la fila 4 nunca se commiteó?

Fila 11 · ¿Cuántos commits de merge?

```
git log --oneline --graph
```

¿Cuántos commits de merge tiene main? ¿Por qué no hay uno para filtro?

Parte 3 · Las comillas

a · Sin comillas dentro del script

Alguien le quita las comillas a la variable en la línea 6 del script ya resuelto: `grep -c MX $archivo`. Luego corre `bash contar.sh "datos/ventas mayo.csv"` sobre un archivo que sí existe con ese nombre. ¿Qué imprime? ¿Qué da echo `$??`? ¿Por qué ese número es peligroso?

b · Sin comillas al llamarlo

Con el script correcto, alguien corre `bash contar.sh datos/ventas mayo.csv`, sin comillas al llamarlo. ¿Qué imprime y por qué?

c · ¿Quién te avisó del conflicto?

En la parte 2, ¿qué herramienta te habría avisado del conflicto antes de construir la imagen de la fila 7: Git, bash o Docker? ¿Con qué comando?
